

COMS 3110 Introduction to the Design and Analysis of Algorithms, Homework 2

Mobina Amrollahi

Due: October 11th, 11:59pm

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions will have to be typed. You can use word, LaTeX, or any other type-setting tool to type your solution. Your submission file should be in pdf format. Do **NOT** submit a photocopy of handwritten homework except for diagrams that can be hand-drawn and scanned. We reserve the right **NOT** to grade homework that does not follow the formatting requirements. Name your submission file: `<Your-net-id>-3110-hw2.pdf`. For instance, if your netid is `asterix`, then your submission file will be named `asterix-3110-hw2.pdf`. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed).

General Requirements

- When proofs are required, do your best to make them both clear and rigorous. Even when proofs are not required, you should justify your answers and explain your work.
- When asked to present a construction, you should show the correctness of the construction.

Some Useful (in)equalities

- $\sum_{i=1}^n i = \frac{n(n+1)}{2}$

- $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
 - $2^{\log_2 n} = n$, $a^{\log_b n} = n^{\log_b a}$, $n^{n/2} \leq n! \leq n^n$, $\log x^a = a \log x$
 - $\log(a \times b) = \log a + \log b$, $\log(a/b) = \log a - \log b$
 - $a + ar + ar^2 + \dots + ar^{n-1} = \frac{a(r^n-1)}{r-1}$
 - $1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
 - $1 + 2 + 4 + \dots + 2^n = 2^{n+1} - 1$
-

1. (10 pts) Prefix-Sum Problem: Given an array A of integers, is there a sub-array of A such that if we add all the elements of the sub-array, we get a sum of zero? By sub-array, we mean a contiguous sub-array.

Naive Brute Force Algorithm (Pseudocode):

```
for i = 1 to n {
    for j = i to n {
        sum = A[i] + A[i + 1] + ... + A[j]
        if sum == 0 return true
    }
}
return false
```

Derive the runtime of the above algorithm in detail and determine its Big-O upper bound. You must show the derivation of the end result. Simply stating the final answer without any derivation steps will result in zero points.

Answer

One important thing to note about the above algorithm is that the term $sum = A[i] + A[i+1] + \dots + A[j]$ cannot be considered an atomic operation. There's a respective loop involved to add elements to the variable sum . Thus, we will change the above algorithm to the following, making sure the rest of the operations in the loop are indeed atomic.

```
for i = 1 to n {
    for j = i to n {
        for k = i to j {
            sum += A[k]
        }
        if sum == 0 return true
    }
}
return false
```

Inner Loop (Controlled by k): Assume the set-up time for the inner loop is c_1 . In addition, since in the loop we do numbers of primitive operations, incrementing sum one single value, we can assume these operations take a constant time c_2 . Therefore:

T_{inner} :

k	Number of Primitive Operations
i	c_2
$i + 1$	c_2
$\vdots$	$\vdots$
j	c_2

$$\Rightarrow T_{inner} = \sum_{k=i}^j c_2 + c_1 = \underbrace{c_2 + c_2 + \cdots + c_2}_{j-i+1 \text{ times}} + c_1 = (j-i+1)c_2 + c_1$$

Middle Loop (Controlled by j): Assume the set-up time for the middle loop is c_3 .
Therefore:

T_{middle} :

j	Number of Primitive Operations
i	$T_{middle} + c_4 = (j-i+1)c_2 + c_1 + c_4$
$i + 1$	$T_{middle} + c_4 = (j-i+1)c_2 + c_1 + c_4$
$\vdots$	$\vdots$
n	$T_{middle} + c_4 = (j-i+1)c_2 + c_1 + c_4$

$$\begin{aligned}
\Rightarrow T_{middle} &= \sum_{j=i}^n ((j-i+1)c_2 + c_1 + c_4) + c_3 \\
&= \sum_{j=i}^n (jc_2) - \sum_{j=i}^n (ic_2) + \sum_{j=i}^n (c_2 + c_1 + c_4) + c_3 \\
&= \sum_{j=i}^n (jc_2) - \sum_{j=i}^n (ic_2) + \underbrace{(c_2 + c_1 + c_4) + \cdots + (c_2 + c_1 + c_4)}_{n-i+1 \text{ times}} + c_3 \\
&= \sum_{j=i}^n (jc_2) - \underbrace{ic_2 + \cdots + ic_2}_{n-i+1 \text{ times}} + (n-i+1)(c_2 + c_1 + c_4) + c_3 \\
&= \sum_{j=1}^n (jc_2) - \sum_{j=1}^{i-1} (jc_2) - (n-i+1)ic_2 + (n-i+1)(c_2 + c_1 + c_4) + c_3 \\
&= \left(\frac{n(n+1)}{2} - \frac{i(i-1)}{2} \right) c_2 - (n-i+1)(ic_2 + c_2 + c_1 + c_4) + c_3
\end{aligned}$$

Outer Loop (Controlled by i): Assume the set-up time for the middle loop is c_5 . Therefore:

T_{outer} :

i	Number of Primitive Operations
1	$T_{middle} + c_6 = (\frac{n(n+1)}{2} - \frac{i(i-1)}{2})c_2 - (n-i+1)(ic_2 + c_2 + c_1 + c_4) + c_3 + c_6$
2	$T_{middle} + c_6 = (\frac{n(n+1)}{2} - \frac{i(i-1)}{2})c_2 - (n-i+1)(ic_2 + c_2 + c_1 + c_4) + c_3 + c_6$
$\vdots$	$\vdots$
n	$T_{middle} + c_6 = (\frac{n(n+1)}{2} - \frac{i(i-1)}{2})c_2 - (n-i+1)(ic_2 + c_2 + c_1 + c_4) + c_3 + c_6$

$$\begin{aligned}
\Rightarrow T_{outer} &= \sum_{i=1}^n ((\frac{n(n+1)}{2} - \frac{i(i-1)}{2})c_2 - (n-i+1)(ic_2 + c_2 + c_1 + c_4) + c_3 + c_6) + c_5 \\
&= \sum_{i=1}^n \frac{n(n+1)}{2}c_2 + \sum_{i=1}^n \frac{i(i-1)}{2}c_2 - \sum_{i=1}^n (n-i+1)(ic_2 + c_2 + c_1 + c_4) \\
&\quad + \sum_{i=1}^n (c_3 + c_6) + c_5 \\
&= \frac{n^2(n+1)}{2}c_2 + \frac{c_2}{2} \sum_{i=1}^n (i^2 - i) - \sum_{i=1}^n (n-i+1)(ic_2 + c_2 + c_1 + c_4) \\
&\quad + n(c_3 + c_6) + c_5 \\
&= \frac{c_2}{2}n^2(n+1) + \frac{c_2}{2} \sum_{i=1}^n (i^2 - i) - (n+1)c_2 \sum_{i=1}^n i + c_2 \sum_{i=1}^n i^2 \\
&\quad - \sum_{i=1}^n (c_2 + c_1 + c_4)(n+1) + (c_2 + c_1 + c_4) \sum_{i=1}^n i \\
&\quad + n(c_3 + c_6) + c_5 \\
&= \frac{c_2}{2}n^2(n+1) + \frac{3c_2}{2} \sum_{i=1}^n (i^2) - (n + \frac{3}{2})c_2 \sum_{i=1}^n i \\
&\quad - (c_2 + c_1 + c_4)(n(n+1)) + (c_2 + c_1 + c_4) \sum_{i=1}^n i
\end{aligned}$$

$$\begin{aligned}
& + n(c_3 + c_6) + c_5 \\
& = \frac{c_2}{2}n^2(n+1) + \frac{3c_2}{2}\sum_{i=1}^n(i^2) - (n + \frac{3}{2})c_2\sum_{i=1}^n i \\
& \quad - (c_2 + c_1 + c_4)(n(n+1)) + (c_2 + c_1 + c_4)(\frac{n(n+1)}{2}) \\
& \quad + n(c_3 + c_6) + c_5 \\
& = \frac{c_2}{2}n^2(n+1) + \frac{3c_2}{2}(\frac{n(n+1)(2n+1)}{6}) - (n + \frac{3}{2})c_2(\frac{n(n+1)}{2}) \\
& \quad - (c_2 + c_1 + c_4)(\frac{n(n+1)}{2}) + n(c_3 + c_6) + c_5 \\
& = \frac{c_2}{2}(n^3 + n^2) + \frac{3c_2}{2}(\frac{2n^3 + 3n^2 + n}{6}) - (n + \frac{3}{2})c_2(\frac{n^2 + n}{2}) \\
& \quad - (c_2 + c_1 + c_4)(\frac{n^2 + n}{2}) + n(c_3 + c_6) + c_5 \\
& = \frac{c_2}{2}(n^3 + n^2) + \frac{3c_2}{2}(\frac{2n^3 + 3n^2 + n}{6}) - \frac{3c_2}{2}(\frac{n^2 + n}{2}) - nc_2(\frac{n^2 + n}{2}) \\
& \quad - (c_2 + c_1 + c_4)(\frac{n^2 + n}{2}) + n(c_3 + c_6) + c_5 \\
& = \frac{c_2}{2}(n^3 + n^2) + \frac{3c_2}{12}(2n^3 + 3n^2 + n) - \frac{3c_2}{4}(n^2 + n) - \frac{c_2}{2}(n^3 + n^2) \\
& \quad - \frac{(c_2 + c_1 + c_4)}{2}(n^2 + n) + n(c_3 + c_6) + c_5 \\
& = (\frac{c_2}{2} + \frac{c_2}{2} - \frac{c_2}{2})n^3 + (\frac{c_2}{2} + \frac{3c_2}{4} - \frac{3c_2}{4} - \frac{c_2}{2} - \frac{(c_2 + c_1 + c_4)}{2})n^2 \\
& \quad + (\frac{c_2}{4} - \frac{3c_2}{4} - \frac{(c_2 + c_1 + c_4)}{2} + c_3 + c_6)n + c_5 \\
& = (\frac{c_2}{2})n^3 - (\frac{c_2 + c_1 + c_4}{2})n^2 - (\frac{c_1 + c_4}{2} + c_3 + c_6)n + c_5
\end{aligned}$$

Since $\frac{c_2}{2} \geq 0$, thus:

$$T_{outer} \in \mathcal{O}(n^3)$$

- (a) **(10 pts)** Given an array of integers, write an efficient algorithm to check whether there is a sub-array whose sum is a multiple of a given integer K . You must use a strategy of computing prefix-sums. Give a detailed analysis of the runtime or expected runtime of your algorithm.

Answer

- i. Description: Knowing that array A and K are our inputs, we will initialize an empty array called "prefixsum," where we will respectively fill out its

elements as we're calculating the prefix sum of the first to i^{th} element of our array A . In addition, we will initialize a hash table, h , which for any positive key " $\text{prefixsum}[i] \bmod k$ ", the value of this key is " i ." Please note the positivity of " $\text{prefixsum}[i] \bmod k$ " is important to have a fair comparison otherwise, although $k-2$ and -2 represent the same remainder of a number to K , differentiating between them, means that they will serve as different keys in our hash table. Further, as we're calculating the elements of $\text{prefixsum}[i]$, we will search for $h[\text{prefixsum}[i] \bmod k]$. If it returns true, that means $\exists j < i$ such that:

$$\text{prefixsum}[j] \equiv \text{prefixsum}[i] \pmod{k}$$

However, we know:

$$\text{prefixsum}[i] = \text{prefixsum}[j] + A[j+1] + A[j+2] + \dots + A[i]$$

Hence:

$$\text{prefixsum}[j] \equiv \text{prefixsum}[j] + A[j+1] + A[j+2] + \dots + A[i] \pmod{k}$$

Which means

$$A[j+1] + A[j+2] + \dots + A[i] \equiv 0 \pmod{k}$$

And in that case, we found $A[j+1 : i+1]$ as a sub-array of A whose sum elements are divisible by K . In this case, we will be returning true since the question is about the existence of such a sub-array. Initially, we set $h[0] = -1$ to handle the scenario where the prefix sum from index 0 to the current index i is divisible by K . This effectively "pretends" that there is a sub-array starting at index -1 so that if we encounter a remainder of 0 at some index i , it means the sum of the sub-array from the start $(-1+1 = 0)$ to i^{th} element is divisible by K .

Finally, we will return false, which means we completely calculated **prefixsum** and yet haven't returned true. Therefore, no such sub-array has been found.

ii. Pseudo code:

Input: Array A of integers, integer K

Initialize an empty hash map $h = \{\}$

Initialize $\text{prefixsum} = []$

Set $h[0] = -1$

$n = \text{size}(A)$

for $i = 0$ to $n - 1$:

$\text{prefixsum}[i] = \text{prefixsum}[i-1] + A[i]$

$\text{division} = \text{prefixsum}[i] \% K$

```

    if division < 0 {
        division = division + K
    } \\endif
    if division in h {
        return True
    } \\endif
    h[division] = i
} \\endfor
return False

```

iii. Run-time Analysis: Initializing an empty hash map, **prefixsum** and setting $h[0] = -1$ take $\mathcal{O}(1)$. Now, consider the loop.

T_{loop} : Assume the setup time for this loop is constant time $\mathcal{O}(1)$. Calculating **prefixsum**[i], **division**, and making sure the division is positive, they all take $\mathcal{O}(1)$. Checking whether **division** is in h is equivalent to searching for it in h , which is **expected** to take $\mathcal{O}(1)$. Finally, calculating $h[\text{division}]$ is **assumed** to take $\mathcal{O}(1)$, meanwhile adding it to the hash table takes $\mathcal{O}(1)$. Thus, one iteration in the whole loop is **expected** to take $\mathcal{O}(1)$. Therefore:

$$\begin{aligned}
 T_{loop} &= \sum_{i=0}^{n-1} (\underbrace{\mathcal{O}(1)}_{\text{initial process in the loop}} \\
 &+ \underbrace{\mathcal{O}(1)}_{\text{expected for "division in h"}} \\
 &+ \underbrace{\mathcal{O}(1)}_{\text{expected for calculating h[division]}} \\
 &+ \underbrace{\mathcal{O}(1)}_{\text{adding h[division]}} + \underbrace{\mathcal{O}(1)}_{\text{set-up time}} \\
 &= (n - 1 - 0 + 1)(4\mathcal{O}(1)) + \mathcal{O}(1) + \\
 &= (n)(4\mathcal{O}(1)) + \mathcal{O}(1) \\
 &= 4n\mathcal{O}(1) + \mathcal{O}(1) \in \mathcal{O}(n)
 \end{aligned}$$

Thus, the **expected** runtime is $\mathcal{O}(n)$.

- (b) **(10 pts)** Prove that the height of a heap tree is $\lfloor \log_2(n) \rfloor$ where n is the number of elements in the tree.

Answer

Suppose the height of the heap is h . Then, a heap with height h has at least 2^h nodes and at most $2^{h+1} - 1$ node. Therefore, n , which is the number of nodes, is

bounded by these two values:

$$\begin{aligned} \Rightarrow 2^h &\leq n \leq 2^{h+1} - 1 \\ \Leftrightarrow \log_2(2^h) &\leq \log_2(n) \leq \log_2(2^{h+1} - 1) \\ \Leftrightarrow \log_2(2^h) &\leq \log_2(n) \leq \log_2(2^{h+1} - 1) < \log_2(2^{h+1}) \\ \Leftrightarrow \log_2(2^h) &\leq \log_2(n) < \log_2(2^{h+1}) \\ \Leftrightarrow h &\leq \log_2(n) < h + 1 \\ \Leftrightarrow h &\leq \log_2(n) < h + 1 \end{aligned}$$

h and $h + 1$ are back to back positive integers, and h is the greatest integer which is less than $\log_2(n)$, which fits the formal definition of $\lfloor \log_2(n) \rfloor$. Hence: $h = \lfloor \log_2(n) \rfloor$.

(c) (10 pts)

- i. (5 pts) Where in a max heap might the smallest element reside, assuming that all elements are distinct?

Answer

The smallest element of a max-heap will be lying in the leaf nodes. To show this, we will prove it by contradiction.

First, let's recall the fundamental property of a max heap, which is that in a max heap, every parent node is greater than or equal to its children.

Prove by Contradiction: For the sake of contradiction, suppose the smallest element of the max-heap A is not a leaf node. Thus, the node i representing this value should be a parent, which means it has at least one child (say, at index $2i$, its left child). Moreover, based on the definition of a max-heap, and if we assume this node is node i , we have:

$$A[i] \geq A[2i]$$

However, we know $A[i]$ is the smallest number within heap A . Hence:

$$A[i] \leq A[2i]$$

Thus:

$$A[i] = A[2i]$$

But this contradicts the assumption that all elements in the heap are distinct because if $A[i] = A[2i]$, the elements are no longer distinct. Thus, the smallest value must lay within the leaf nodes.

- ii. (5 pts) Is an array that is sorted in increasing order a min heap? Justify your answer with an explanation.

Answer

Yes. Suppose array A is in ascending order and $|A| = n$. Then:

$$\forall i, j \leq n \text{ and } i \leq j : A[i] \leq A[j]$$

There will be three possible outcomes $\forall i \leq n$

- A. $2i + 1 \leq n$. $2i$ and $2i + 1$ are the indices of the left and right children, respectively. Thus:

$$i < 2i \Rightarrow A[i] \leq A[2i]$$

$$i < 2i + 1 \Rightarrow A[i] \leq A[2i + 1]$$

Therefore, in this case, a parent node dominates both its children by containing a smaller key value than they do.

- B. $2i < n$ and $2i + 1 > n$. Then the parent node i has only one left child, which is node $2i$. Then:

$$i < 2i \Rightarrow A[i] \leq A[2i]$$

Therefore, in this case, a parent node dominates its only left child by containing a smaller key value than they do.

- C. $2i > n$. The node i is a leaf node without any child. Thus, there's no need for node i to dominate any other nodes.

These three cases cover all possible scenarios for any node in the array, and in all of them, they obey the min-heap structure. Hence, Array A is a min-heap.

- (d) **(10 pts)** Consider an array H of size $n \geq 2$, containing distinct positive integers and an integer K such that $1 \leq K \leq n$. Without sorting the array, write an algorithm with input H and K to find the K^{th} largest integer in H . Your algorithm must use a heap data structure. Write pseudocode for your algorithm and also present an analysis of its correctness and the runtime in terms of K and n .

Answer

- i. Description: to find the K^{th} largest element in an unsorted array H of distinct integers using a heap data structure, we will use a max-heap, knowing that the largest element is always at the root of such heap.

The algorithm starts by building a max-heap from the array H , which takes $\mathcal{O}(n)$ time. Once the max heap is built, the root element (which is the largest) is swapped with the last element of the heap, and the heap size is reduced

by 1. This process will be repeated K times. After $K - 1$ rounds, the root element will be the K^{th} largest element in the array.

In contrast to the full heapsort algorithm, which sorts the entire array, we stop the process early after extracting the K^{th} largest element.

ii. Pseudo code:

```

Input: Array H of integers, integer K

BuildMaxHeap(H)
for i = len(H) down to len(H) - K + 1 {
    swap(H[1], H[i])
    decrement heap size by 1
    HeapifyDown(H, 1)
}
return H[1]

```

iii. Run-time analysis: Building the max heap will take $\mathcal{O}(n)$ times. For the loop, the set-up time is $\mathcal{O}(1)$. Swapping $H[1]$ and $H[i]$, as well as decrementing the size of the heap, takes $\mathcal{O}(1)$. Finally, heapifying down will take $\mathcal{O}(\log n)$. Hence:

$$\begin{aligned}
 T_{\text{inner}} &= \sum_{i=\text{len}(H)-K+1}^{\text{len}(H)} (\mathcal{O}(1) + \mathcal{O}(\log n)) + \mathcal{O}(1) \\
 &= (\text{len}(H) - (\text{len}(H) - K + 1) + 1)(\mathcal{O}(1) + \mathcal{O}(\log n)) + \mathcal{O}(1) \\
 &= K(\mathcal{O}(1) + \mathcal{O}(\log n)) + \mathcal{O}(1)
 \end{aligned}$$

Therefore, the total runtime is:

$$\mathcal{O}(n) + K \times \mathcal{O}(\log n) + (K + 1)\mathcal{O}(1) \in \mathcal{O}(n + K \log n)$$

Depending on the bound on K in terms of n , the time complexity have two possibilities:

$$\text{Time complexity} = \begin{cases} \mathcal{O}(n) & \text{if } K \leq \frac{n}{\log n} \\ \mathcal{O}(K \log n) & \text{if } \frac{n}{\log n} < K \leq n \end{cases}$$

Hence:

$$\text{Time complexity} = \mathcal{O}(n + \min(K \log n, n \log n))$$

List of Collaborators

Ryan Holt and I discussed our solutions to questions 1, 4 and 5.

In question 1, we discussed whether we need to consider the summation inside the loop

as a separate loop or not.

In question 4, I mentioned to Ryan how the distinction of the variables in the heap is important.

Finally, in question 5, we discussed what level of detail our algorithms should be. Finally, I decided to leave the `BuildMaxHeap` and `HeapifyDown` as mentioned, and Ryan explained it in his code.